

GOVERNMENT OF TAMIL NADU

**NAAN MUDHALVAN SKILL DEVELOPMENT
PROGRAMME**

A PROJECT REPORT

submitted under

NAAN MUDHALVAN SKILL DEVELOPMENT PROGRAMME
Government of Tamil Nadu

PROJECT TITLE:

LegalEase - AI-Powered Legal Document Generator

TEAM ID	: SWTID-2026-1904
Team Size	: 4
TEAM LEADER	: Thahsina M
TEAM MEMBERS	: Kalaiyarasi P, Bhuvaneshwari K,
	Reehana Mohammed Iqbal

ACADEMIC YEAR : 2026-2027

**SKILL DEVELOPMENT PROGRAMME | DEPARTMENT OF EMPLOYMENT & TRAINING | GOVERNMENT
OF TAMIL NADU**

TABLE OF CONTENTS

1. Abstract
2. Introduction
3. Objectives and Scope
4. Technology Stack and Requirements
5. System Architecture
6. Project Workflow and Modules
7. Implementation
8. User Guide and Output
9. Testing and Results
10. Deployment
11. Conclusion
12. Future Enhancements
13. References

1. Abstract

LegalEase is an AI-powered web application that generates professional legal documents such as freelance contracts, non-disclosure agreements, lease agreements and employment offer letters. The user enters four inputs - document type, parties involved, terms and conditions, and effective date - and the system produces a complete, formally structured legal document using Google's Gemini 2.5 Flash model.

The application uses Streamlit as the frontend, FastAPI as the backend, and the Google Generative AI SDK as the AI core. Generated documents can be previewed in the browser, edited inline, and exported as .txt, .docx and .pdf files with logo, headings, terms table and footer. LegalEase makes basic legal drafting accessible to entrepreneurs, freelancers, landlords and individuals who do not have a legal background.

2. Introduction

2.1 Background

Drafting a legal document normally requires a lawyer or costly templates. For small businesses, freelancers and individuals this is slow and expensive. Recent Generative AI models can understand structured instructions and write long, well-organised formal text, which makes them suitable for drafting first versions of standard legal documents.

2.2 Problem Statement

Users without legal knowledge find it difficult to create contracts and agreements that follow a proper structure, include the right clauses, and are available in usable formats. Existing templates are static and do not adapt to the parties, terms and dates of each case.

2.3 Proposed Solution

LegalEase takes the user's inputs, builds a structured prompt, sends it to Gemini 2.5 Flash through a FastAPI backend, and returns a complete document. The user can edit the text and download it in the required format with custom branding.

3. Objectives and Scope

3.1 Objectives

- Select a suitable generative AI model and define the system architecture.
- Generate customised legal documents from user inputs.
- Provide an editable preview of the generated document.
- Export documents as .txt, .docx and .pdf with logo, terms table and footer.
- Build a modular FastAPI backend and a user-friendly Streamlit frontend.
- Deploy and test the complete application end to end.

3.2 Scope

The system supports employment contracts, lease agreements, NDAs, freelance work contracts and similar agreements. It is intended as a drafting aid; generated documents should be reviewed by a qualified legal professional before they are signed or relied upon.

3.3 Usage Scenarios

- Scenario 1: A startup founder enters a new hire's details, selects Employment Contract, edits the wording and downloads a branded PDF.
- Scenario 2: A freelancer specifies the parties, scope of confidentiality and effective date to instantly receive an NDA.
- Scenario 3: A landlord enters property address, tenant details and lease terms and downloads a residential lease agreement as .docx.

4. Technology Stack and Requirements

Component	Technology
Frontend	Streamlit
Backend	FastAPI with Uvicorn server
AI Model	Google Gemini 2.5 Flash (google-generativeai SDK)
Data validation	Pydantic
DOCX export	python-docx
PDF export	fpdf
Images / logo	Pillow
HTTP client	requests
Configuration	python-dotenv (.env file for API key)
Language	Python 3.10+

4.1 Environment Setup

```
python -m venv venv
venv\scripts\activate
pip install fastapi uvicorn streamlit python-docx fpdf Pillow requests google-generativeai python-dotenv
```

A .env file stores the Gemini API key (GEMINI_API_KEY) so that the key is never written inside the source code.

5. System Architecture

LegalEase follows a three-layer architecture. The Streamlit frontend collects user input and shows the preview and download options. The FastAPI backend receives a JSON request, validates it and calls the AI generator. The AI core (gemini_generator.py) builds a structured prompt and sends it to the Gemini API, then returns the generated text. Formatting modules convert the text into .txt, .docx and .pdf outputs.

User Interface (Streamlit) → FastAPI Backend (main.py, routes.py) → Document Generator (gemini_generator.py) → Gemini Google API → Formatted Output (.txt, .docx, .pdf)

Figure 5.1: System architecture and data flow

5.1 Project Folder Structure

```
LEGALEASE/  
  ai_core/  
    gemini_generator.py  
    generator.py  
  docs/  
  frontend/  
    app.py  
  Image/  
    Logo.png, inverseLogo.png  
  legalEaseAPI/  
    main.py  
    routes.py  
  venv/  
  .env  
  config.py  
  requirements.txt  
  run.sh
```

6. Project Workflow and Modules

Milestone 1: Model Selection and Architecture

The project requirements (document type, parties, terms and effective date) were analysed and the Gemini model documentation was reviewed. gemini-2.5-flash was selected because of its long context window (up to 1 million tokens), support for structured multi-part prompting, fast generation, and strong reasoning and formatting ability needed for formal legal text. The architecture and development environment were then defined.

Milestone 2: Core Functionalities Development

- Legal document generation from document type, parties, terms and dates.
- Document output formatting through `format_docx()`, `format_pdf()` and `format_html_preview()`.
- User interaction through Streamlit input fields.
- Editable text area and multi-format downloads.

Milestone 3: API Logic Integration

`main.py` initialises the FastAPI app, exposes a root (`/`) endpoint as a health check and includes the router from `routes.py`. `routes.py` defines the `DocumentRequest` Pydantic model and the `POST /generate` endpoint that calls `GeminiDocumentGenerator`.

Milestone 4: Frontend Development

The Streamlit interface uses a centered layout with logo and title, four input fields, a Generate Document button, a dark-themed HTML preview card, an Edit Document option and three download buttons.

Milestone 5: Deployment

The backend and frontend were deployed locally using a virtual environment, `requirements.txt` and a `.env` file, and all functions were tested together.

7. Implementation

7.1 main.py

```
from fastapi import FastAPI
from legalEaseAPI.routes import router

app = FastAPI(title="LegalEase - AI Legal Document Generator")
app.include_router(router)

@app.get("/")
def home():
    return {"message": "Welcome to LegalEase AI Legal Document Generator API"}

if __name__ == "__main__":
    import uvicorn
    uvicorn.run(app, host="0.0.0.0", port=8000)
```

7.2 routes.py

```
from fastapi import APIRouter
from pydantic import BaseModel
from ai_core.gemini_generator import GeminiDocumentGenerator

router = APIRouter()
gemini_generator = GeminiDocumentGenerator()

class DocumentRequest(BaseModel):
    document_type: str
    parties: str
    terms: str
    dates: str

@router.post("/generate")
def generate_legal_document(request: DocumentRequest):
    response = gemini_generator.generate_document(
        request.document_type, request.parties,
        request.terms, request.dates)
    return {"document": response}
```

7.3 gemini_generator.py

```
genai.configure(api_key=os.getenv("GEMINI_API_KEY"))
self.model = genai.GenerativeModel(model_name)

def generate_document(self, document_type, parties, terms, dates):
    prompt = (
        f"Generate a comprehensive legal document titled '{document_type}'\n"
        f"Involved parties: {parties}\n"
        f"Effective Date: {dates}\n"
        f"Terms and conditions: {terms}\n"
        f"Ensure formal legal structure with multiple sections and legal clauses."
    )
    response = self.model.generate_content(prompt)
    return response.text
```

7.4 Frontend (app.py)

```
st.set_page_config(page_title="LegalEase", layout="centered")
document_type = st.text_input("Document Type")
parties = st.text_area("Parties Involved")
```

```

terms = st.text_area("Terms & Conditions (Use semicolons for bullet points)")
dates = st.text_input("Effective Date")

response = requests.post("http://localhost:8000/generate", json={
    "document_type": document_type, "parties": parties,
    "terms": terms, "dates": dates})
generated_text = sanitize_text(response.json()["document"])

```

7.5 Core Utility Functions

Function	Purpose
<code>sanitize_text(text)</code>	Removes special characters and typographic quotes for clean formatting.
<code>format_docx(text, doc_type)</code>	Uses python-docx to embed the logo, add titles and paragraphs, and auto-generate a Terms table from semicolon-separated input.
<code>format_pdf(text, doc_type)</code>	Uses FPDF with custom header and footer, centered logo, bold section headings and bullet-style terms.
<code>format_html_preview(text)</code>	Converts output into styled HTML blocks for inline preview.

8. User Guide and Output

8.1 Input Fields

Field	Purpose	Example
Document Type	Type of legal document	Freelance Work Contract
Parties Involved	Names and roles of parties	Jane Doe (Service Provider), TechNova Inc. (Client)
Terms & Conditions	Clauses separated by semicolons	Work delivered by May 15, 2025; Payment within 7 days of invoice
Effective Date	Date the agreement becomes valid	April 15, 2025

8.2 Generating, Editing and Downloading

- Click Generate Document; a success message and the dark-themed preview appear.
- Click Click to Edit Document to open an editable text area; press Ctrl+Enter to apply changes.
- Download as .TXT (plain text), .DOCX (logo on front page, headings, terms table, footer on last page) or .PDF (logo and footer on every page).

[Insert screenshots here: Figure 8.1 Home page of LegalEase; Figure 8.2 Filled input form; Figure 8.3 Generated document preview; Figure 8.4 Edit mode; Figure 8.5 TXT output; Figure 8.6 DOCX output; Figure 8.7 PDF output]

9. Testing and Results

The application was tested end to end using different input combinations. Each test confirmed that the backend returned a complete document and that all three export formats opened correctly.

No.	Test Case	Expected Result	Status
1	Start FastAPI with uvicorn; open /	Welcome message returned	Pass
2	Generate Freelance Work Contract	Structured contract with all inputs	Pass
3	Generate NDA with different parties	Document reflects parties and date	Pass
4	Terms separated by semicolons	Terms appear as bullets / table	Pass
5	Edit text and apply	Edited text used in downloads	Pass
6	Download .txt	Plain text file opens	Pass
7	Download .docx	Logo, headings, table, footer present	Pass
8	Download .pdf	Logo and footer on all pages	Pass

Result: LegalEase successfully generates formatted, editable legal documents from simple user inputs within seconds.

10. Deployment

The application runs locally with two commands from the project folder, with the virtual environment activated and the Gemini API key stored in .env.

```
uvicorn legalEaseAPI.main:app --reload      # backend on http://127.0.0.1:8000
streamlit run frontend/app.py              # frontend on http://localhost:8501
```

For cloud deployment, requirements.txt, a Procfile or a Dockerfile can be added. The frontend can be hosted on Streamlit Community Cloud and the backend on Render, Railway, Fly.io or a VPS.

11. Conclusion

LegalEase demonstrates how Generative AI can bridge the accessibility gap in legal services. By combining Gemini 2.5 Flash with a FastAPI backend and a Streamlit interface, the project lets users create customised legal documents, edit them and export them in professional formats without legal expertise. The modular architecture is scalable and easy to maintain, and the project met all of its milestone objectives.

12. Future Enhancements

- Deeper contract analysis and plain-language summaries of clauses.
- Multilingual support, including Tamil, for wider reach.
- Integration with legal databases and e-signature services.
- User accounts with saved documents and templates.
- Personalised clause recommendations and cloud deployment.

13. References

- FastAPI Documentation - <https://fastapi.tiangolo.com>
- Google Generative AI Documentation - <https://ai.google.dev>
- Streamlit Documentation - <https://docs.streamlit.io>
- Python Official Documentation - <https://docs.python.org>
- SmartBridge / SmartInternz LegalEase project guide.